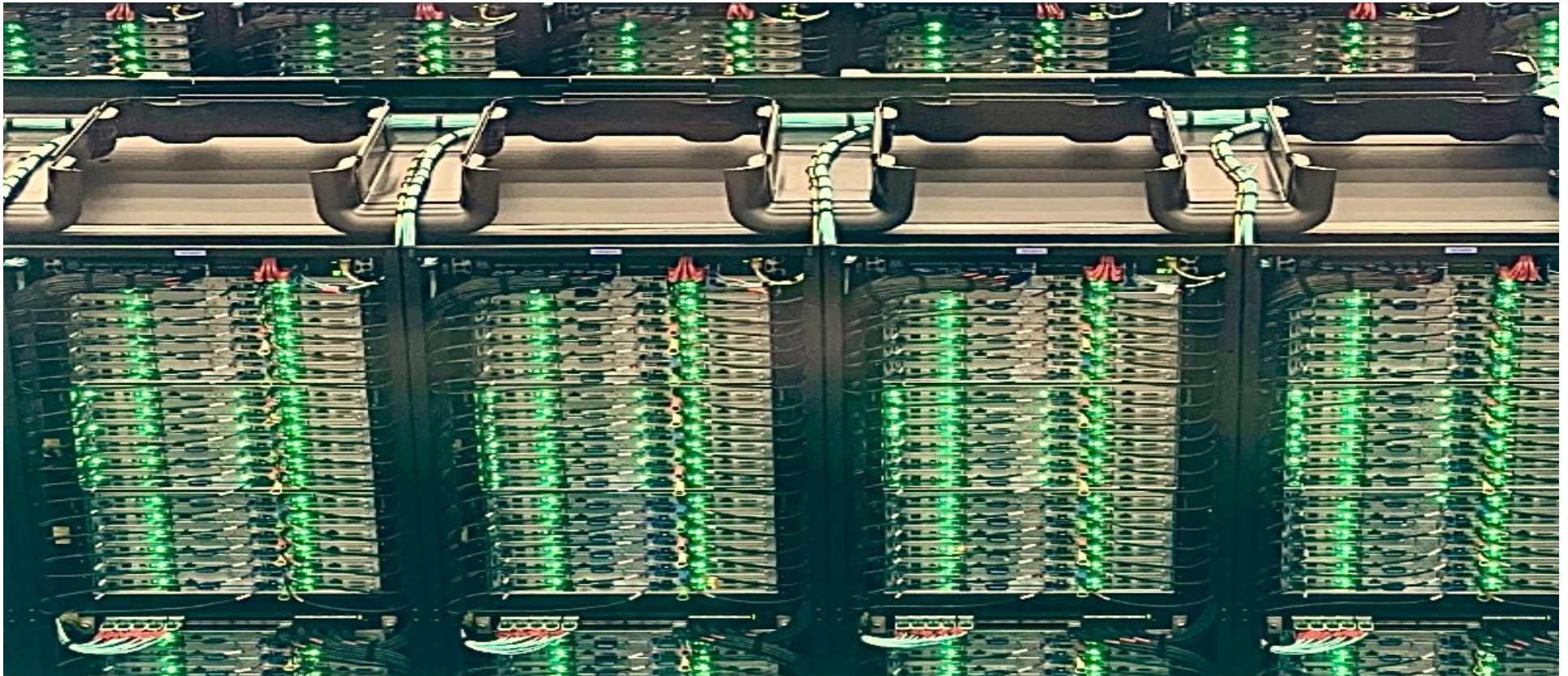


01. Supercomputing Basics

Supercomputing for Artificial Intelligence
Foundations, Architectures, and Scaling Deep Learning Workloads

Jordi **TORRES.AI**





What is Supercomputing?

Supercomputing

■ Context

- AI has transformed supercomputing
- Classical foundations remain essential
- Book combines fundamentals + new AI-driven topics

■ Definition

- Supercomputer = system far beyond desktops/laptops
- Definition is relative (today's laptop > past supercomputers)
- Need for ever more powerful systems persists

Supercomputing

■ Origins

- 1960s: Seymour Cray (Control Data Corporation)
- First machines designed for high performance tasks
- Now essential for science and engineering
- Evolution tracked by Top500 list (updated twice/year)

■ Core principle: Parallel computing

- Executes many operations simultaneously
- Workload distributed across thousands/millions of units
- Central to modern supercomputing

High Performance Computing vs. Supercomputing

■ Impact

- Transformative technology across science and practical challenges (e.g. AI)
- Progress unmatched in computing history
 - 1940s: < 1,000 ops/sec
 - Today: > 1 exaFLOP (≈ 1.7 exaFLOPS top systems)
- Driven by innovation in hardware, architectures, programming, algorithms, software

■ Terminology?

- Supercomputing → use of powerful specialized machines
- HPC → broader: hardware + software + programming + resource management + algorithms
- **Terms often used interchangeably**
 - “Supercomputing” resonates more with students and general audiences

High Performance Computing vs. Supercomputing

- **HPC as a discipline (more precise academic term)**
 - Goal: maximize computing capability with available tech
 - Dynamic, adapts to challenges and paradigms
- **Three pillars:**
 - **Technology:** CPUs, GPUs, memory, interconnects, storage
 - **Methodology:** parallelization, coordination, scaling
 - **Application:** science & industry (climate, genomics, fluid dynamics, etc.)
- **Convergence of these three pillars + AI**
 - HPC as a foundational discipline for modern research and innovation

Parallelism: The Cornerstone of HPC

■ Why Parallel Computing?

- Until ~2010: performance improved by higher clock frequency (60 MHz → ~5 GHz)
- Physical limits: miniaturization, power, heat → frequency scaling stalled
- Solution: parallelism with multiple moderately fast units
- More scalable and energy-efficient → cornerstone of modern HPC

■ **Parallelism = divide task into smaller parts, executed simultaneously**

Serial vs Parallel Execution

■ Serial Execution on Standard Computers

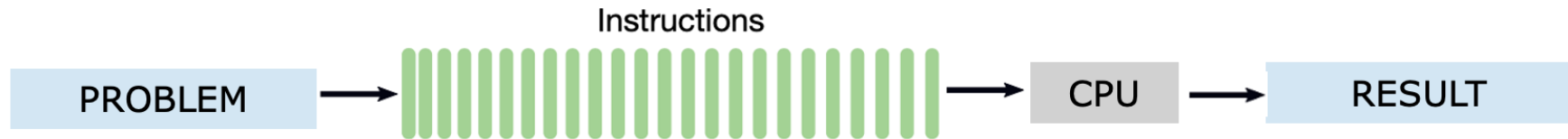
- Instructions executed sequentially on a single unit
- Adequate for everyday tasks (web, office, small programs)
- Limitations for large-scale workloads:
 - Single CPU speed, memory, throughput insufficient
 - Bottleneck for simulations, big data, deep learning training

■ Parallel Execution in HPC Systems

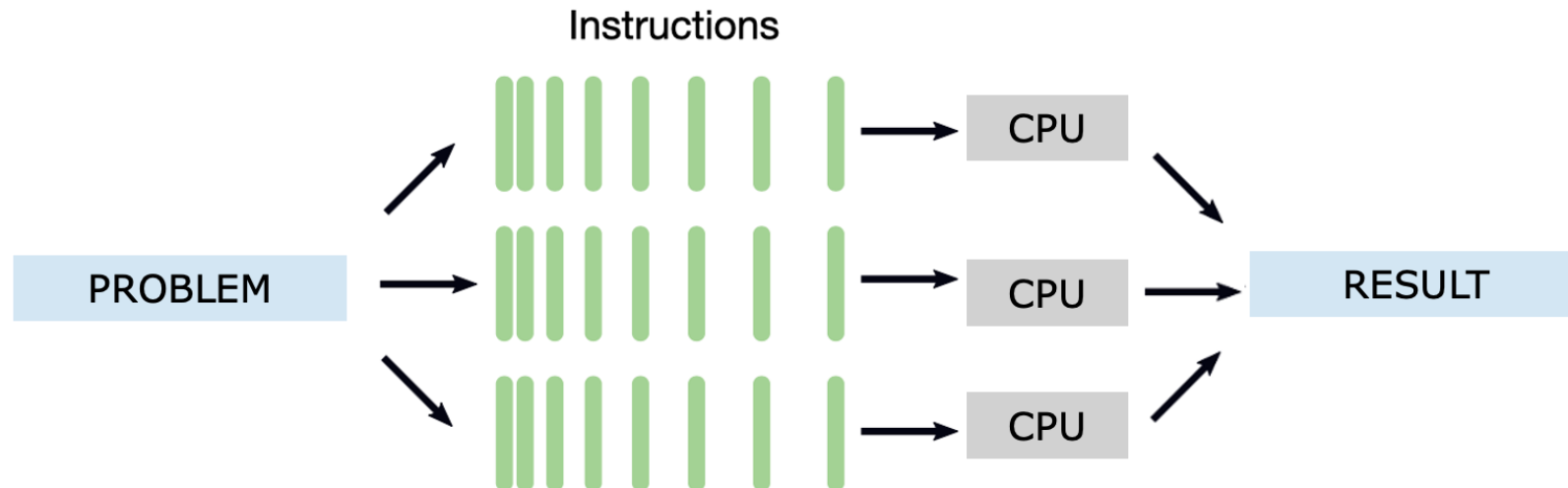
- Benefits: faster, scalable in speed and memory capacity
- Added complexity: task assignment, synchronization, communication
- Motivations:
 - **Reduce execution time** (e.g. weather forecasting, epidemiology)
 - **Solve problems too large for a single machine** (climate, AI models, genomics)

Sequential vs Parallel Execution

- **Sequential***: single stream of instructions on one CPU



- **Parallel**: problem divided into independent sub-tasks on multiple CPUs/nodes



(*) Serial vs Sequential

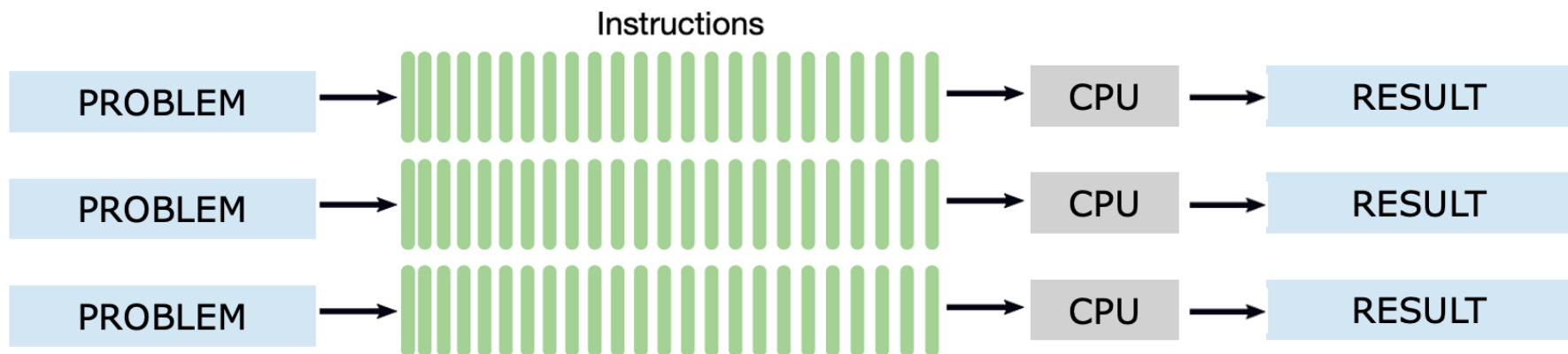
Clarification of terms:

- ***Sequential***: instructions executed one after another, even if parallelism possible
- ***Serial***: inherently non-parallelizable portion of a program
- **In this course**: terms used interchangeably unless precise distinction needed

Parallelism vs Concurrency

■ Concurrency in HPC Systems

- Execution of multiple independent problems in parallel
- Each problem → own instruction stream + dedicated CPU/core
- No need for communication/synchronization
- Maximizes resource utilization
- Typical use cases: hyperparameter tuning, multi-user jobs



Limitations and Challenges of Parallelization

- **Not all problems parallelize well**
- **Main issues:**
 - **Task dependencies** → force sequential execution
 - **Imbalanced workloads** → idle processors, poor efficiency
 - **Redundant computations** → overhead, memory/data bottlenecks
- **Challenge: efficient, scalable, sustainable parallelization**
(topics covered later in the course, core content)
- **Requires careful algorithm design + specialized programming models**



Parallel Computer Architecture basics

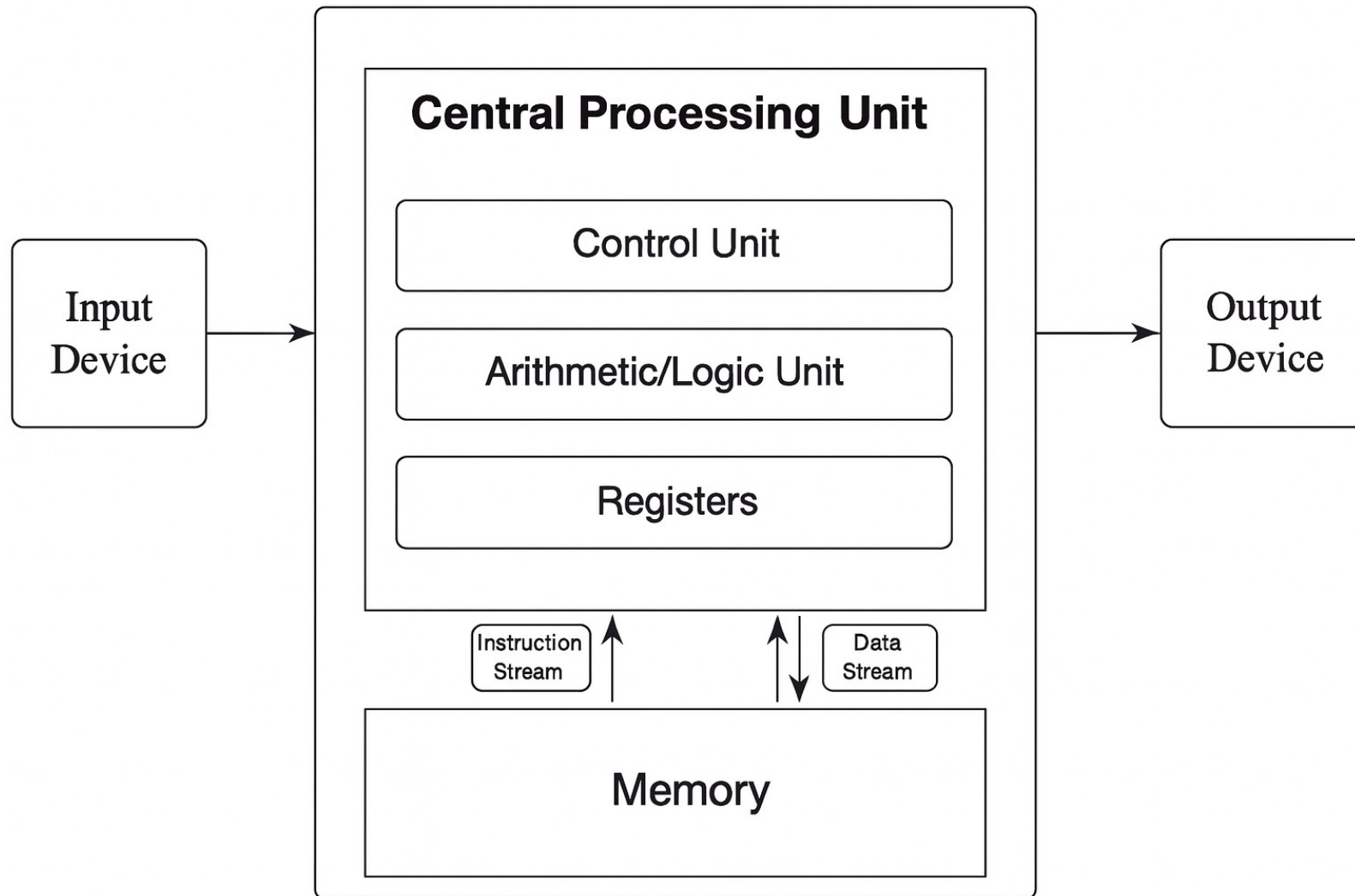
Understanding Parallel Computer Architectures

- To understand parallel computing, we must study computer architectures
- Historical: von Neumann model
- Modern: multi-CPU and GPU systems in distributed environments
- Goal: understand performance characteristics and limitations of today's supercomputers

The Von Neumann Model

- Introduced in the 1940s
- Conceptual backbone of modern computing
- Single processing unit executes instructions sequentially from shared memory
- **CPU components:**
 - Control Unit (orchestrates execution)
 - Arithmetic/Logic Unit (basic operations)
 - Registers (fast temporary storage)
- **Also includes memory + input/output devices**
- **Still relevant as the foundation for complex architectures**

The Von Neumann Model



From Single to Multiple Cores

- **Demand exceeded single CPU capability**
- **Solution? multiple cores per chip**
- **Modern CPUs: up to 64 cores per socket**
 - Distinction:
 - CPU = physical package
 - Core = processing unit inside
 - Programming perspective: each core = independent processor

Memory Architectures for Parallelism

- **Shared Memory (UMA)**

- All CPUs access same memory with equal latency
- Simplifies programming, but contention limits scalability

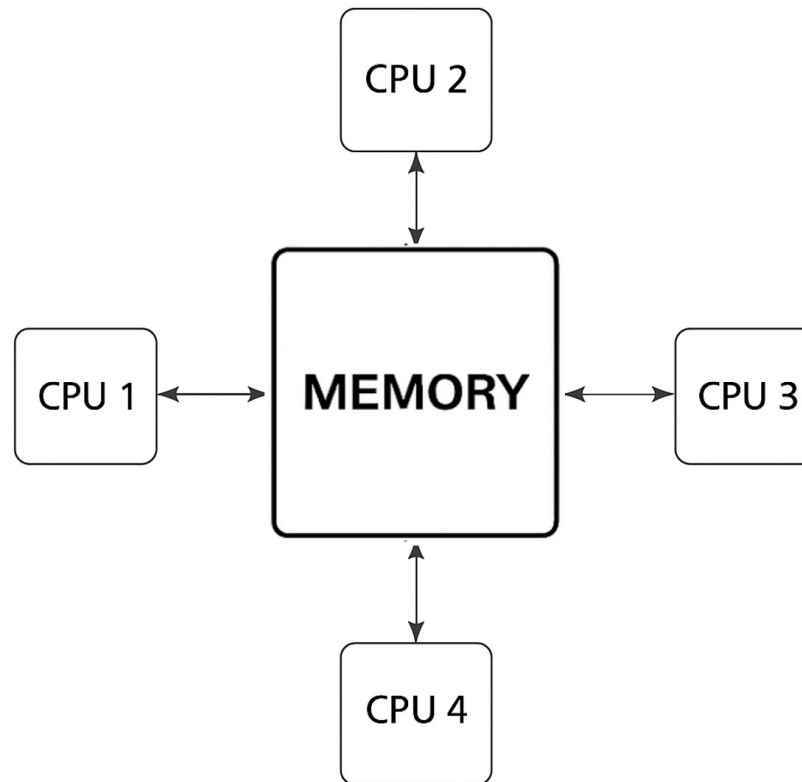
- **NUMA (Non-Uniform Memory Access)**

- Each CPU accesses local memory faster
- Can still reach remote memory, but with higher latency
- Reduces contention, improves scalability, adds complexity

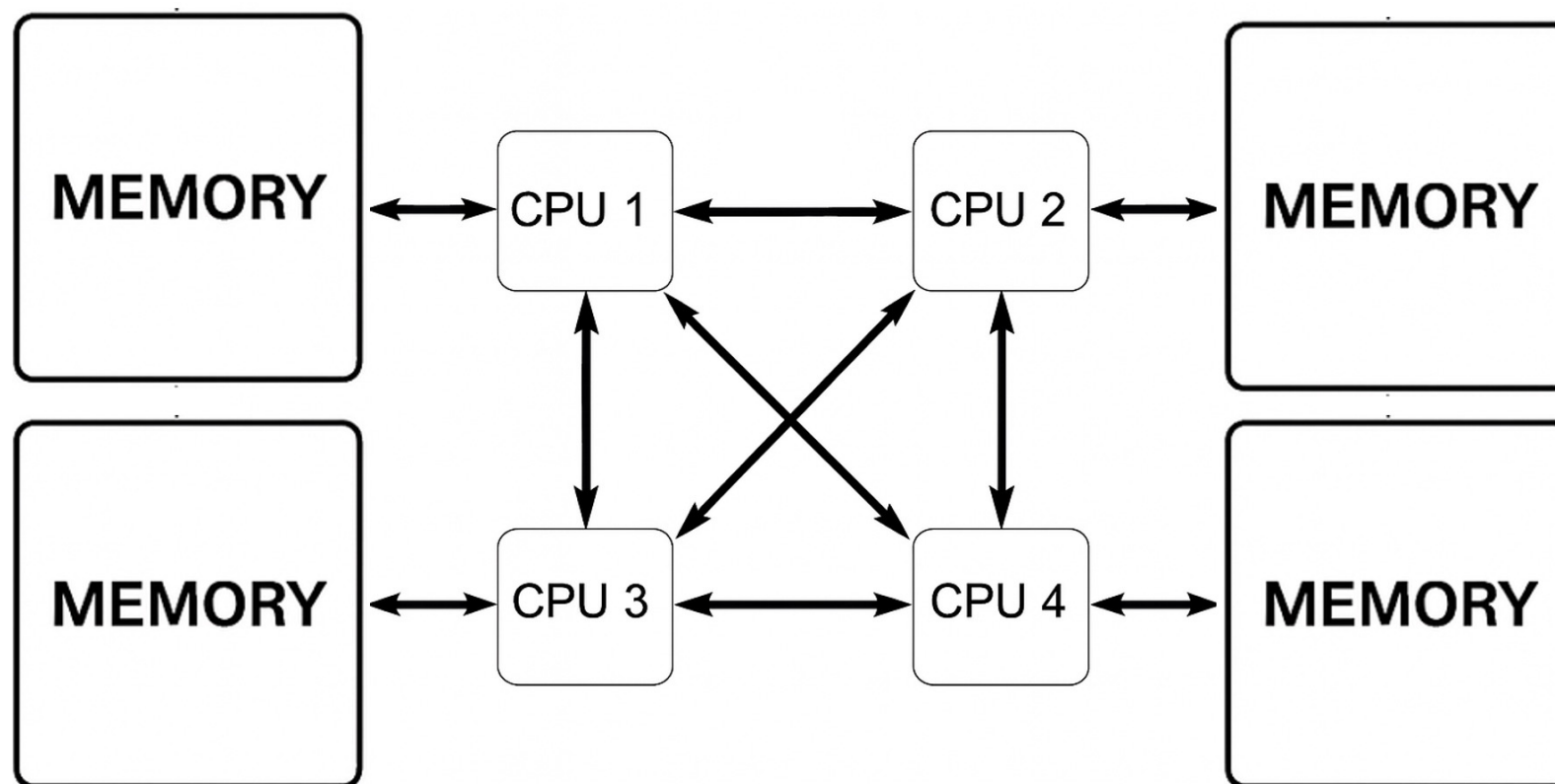
- **Distributed Memory**

- Each CPU has private memory
- Communication via message passing over high-speed network
- Highly scalable, but programming more complex

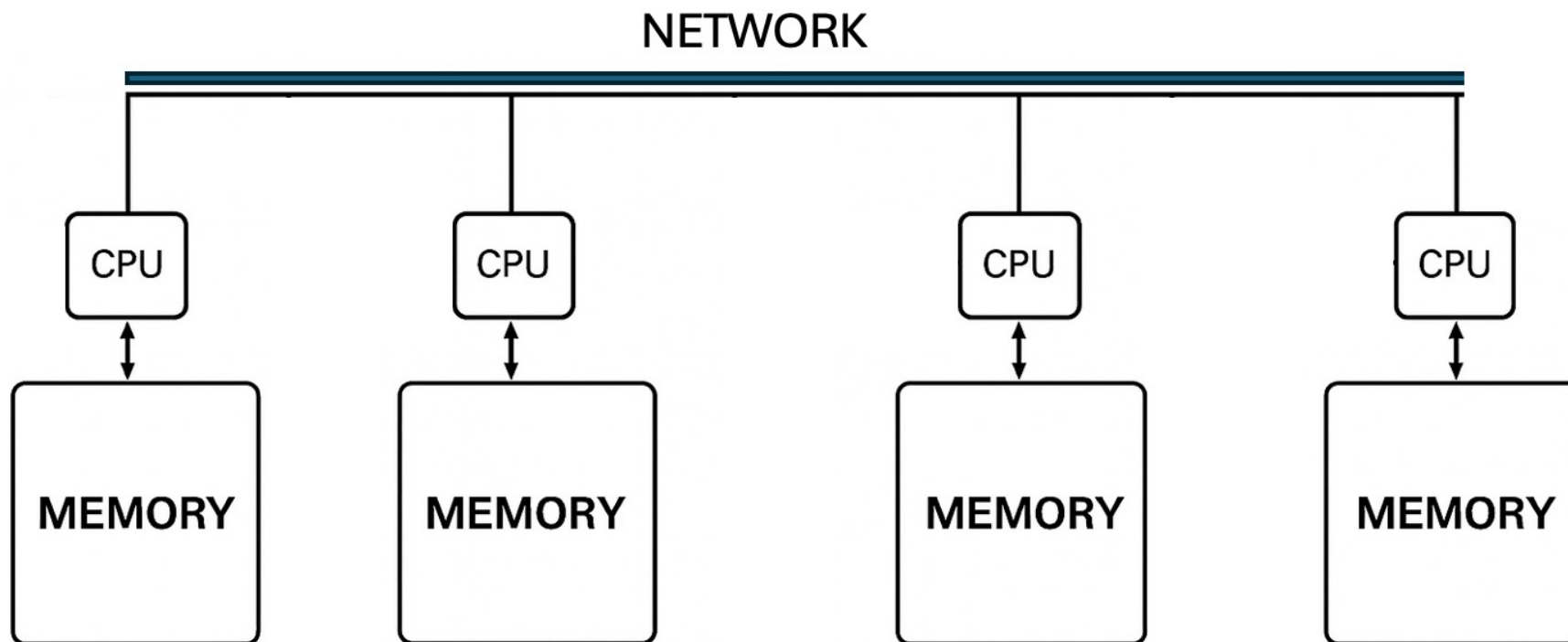
Shared Memory (UMA)



NUMA (Non-Uniform Memory Access)

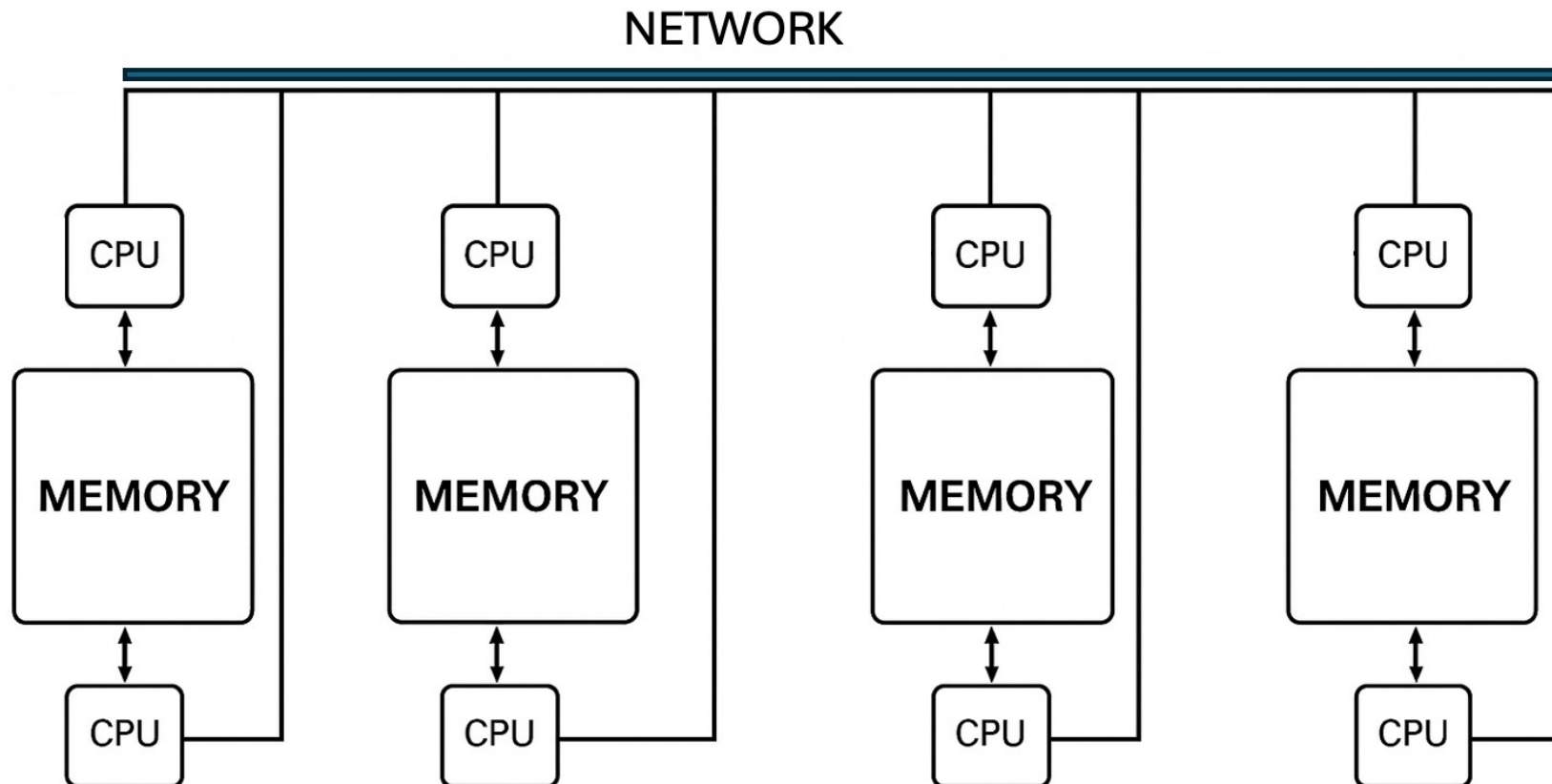


Distributed Memory



Hybrid Shared + Distributed Memory

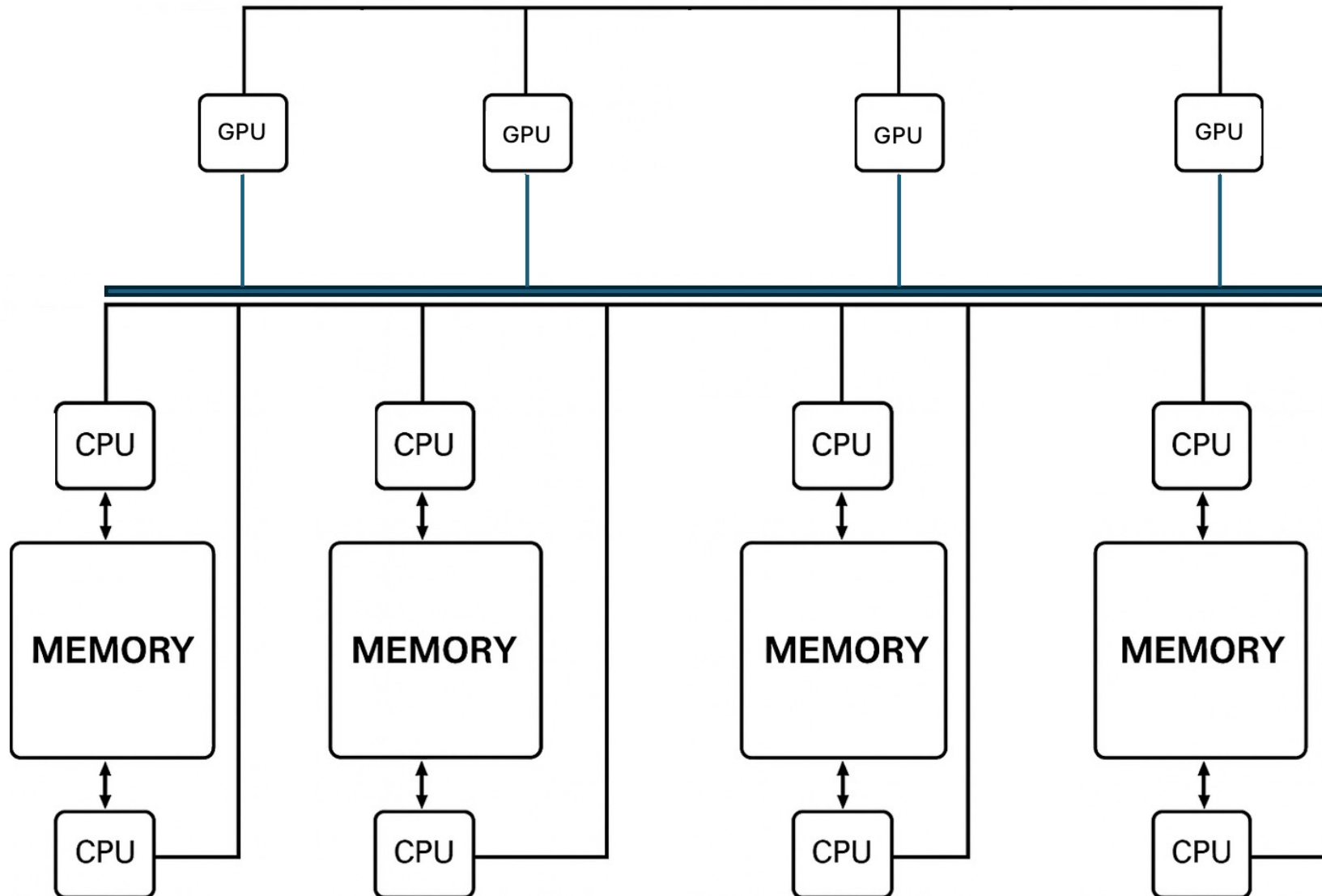
- Dominant model in modern HPC
- Within node: CPUs share memory
- Across nodes: connected by network



Adding GPUs to the Architecture

- **GPUs: originally for graphics, now key in HPC/AI**
- **GPU?**
 - Thousands of simple cores, optimized for throughput
- **Complement CPUs:**
 - CPUs → control, serial execution
 - GPUs → high-throughput parallelism
- **GPUs act as co-processors, orchestrated by CPUs**
- **Modern supercomputing nodes:**
 - Multiple CPUs + GPUs
 - Local memory per device
 - High-speed interconnect across nodes

Hybrid CPU-GPU architecture with distributed nodes





How Supercomputing Powers Modern Science

Classical Paradigm

- **Discovery based on experiment + theory**

- **Cycle:**

observation → hypothesis → experiment → refinement

- **Limitations:**

- Some phenomena are too dangerous (nuclear fusion)
- Some phenomena are too expensive (jet crash tests)
- Some phenomena are too slow (climate change)
- Some phenomena are too vast/small to manipulate physically

- **Need for a new tool**

→ simulation

Computational Science (Third Paradigm)

- Emerged with computers, supercomputers since 1990s
- Creates virtual models of reality → test hypotheses computationally
- Key in fields as:
 - astrophysics, materials science, engineering, ...
- Enables discovery when experiments are impractical or impossible
- Digital twins: simulate prototypes before physical builds → virtual laboratories

Big Data-Driven Science (Fourth Paradigm)

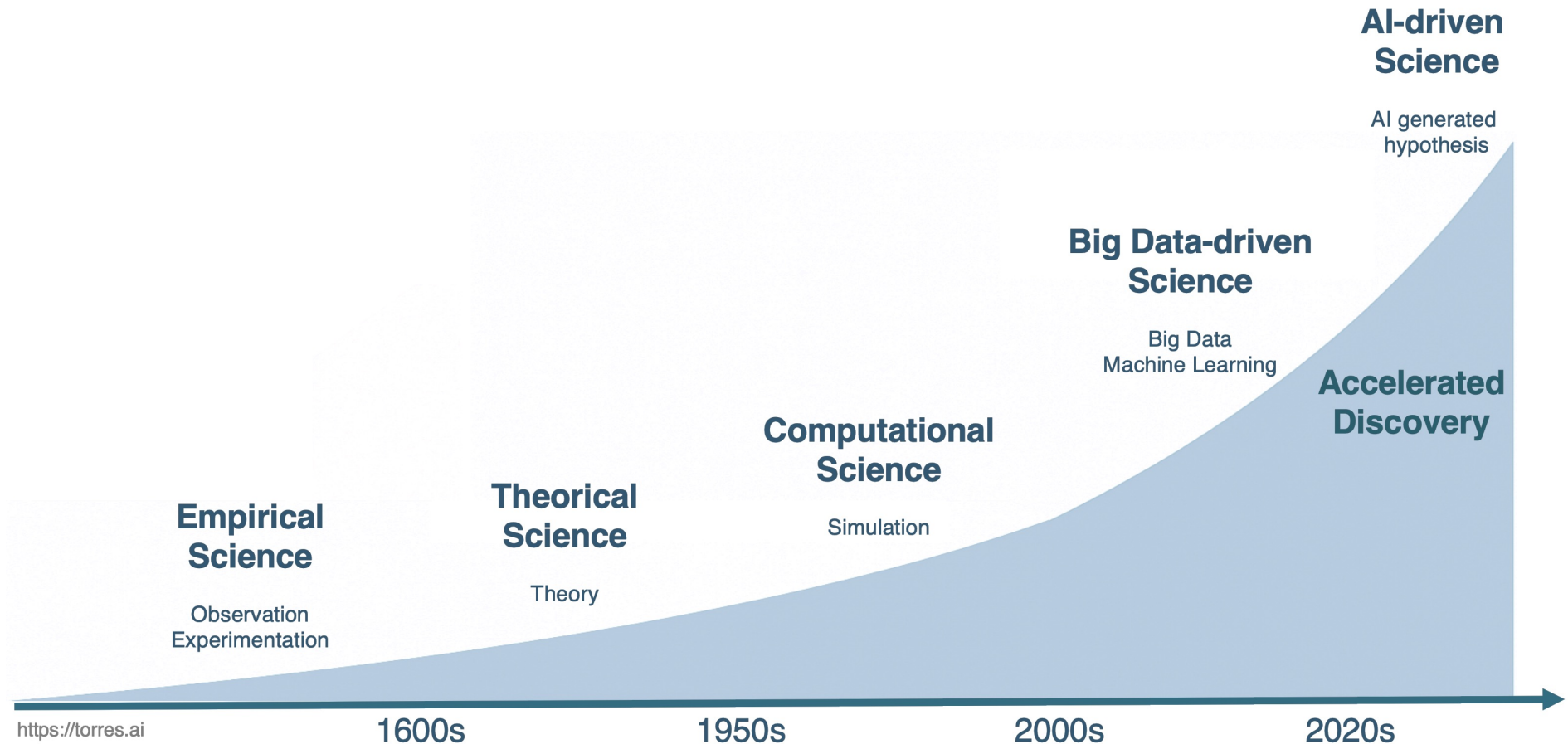
- **Explosion of data (21st century)**
 - genomics, astrophysics, etc.
- **Use of machine learning, pattern recognition, ...**
- **Insights extracted from massive datasets**
- **Science shifts:**

→ laws approximated from empirical data
- **Effective for large, complex, noisy datasets**

AI-Driven Science (Fifth Paradigm)

- **Builds on big data paradigm**
- **AI goes beyond analysis:**
 - Generates hypotheses
 - Designs experiments
 - Optimizes research strategies
 - ...
- **Accelerates and automates discovery**
- **AI as autonomous collaborator with scientists**
- **Aligned with modern supercomputing: training powerful AI models at scale**

Evolution of scientific paradigms throughout the history of science



- Current frontier: AI + supercomputing = accelerated discovery
- Profound transformation: faster, automated, scalable science



Performance in Supercomputing

Performance and Metrics

- **Performance = not a single number → multiple metrics**
- **Metric = observable, quantifiable operational parameter**
- **Two basic dimensions:**
 - Time to complete a task
 - Number of operations performed
- **FLOPS (Floating-Point Operations Per Second):**
 - Measures arithmetic operations/sec
 - Hardware-independent comparison

Performance and Metrics

- **FLOPS = throughput → rate of useful operations per time unit**
- **Scientific prefixes used for scale**

Prefix	Abbreviation	Order Magnitude	Computer Performance	Performance Abbreviation
giga	G	10^9	gigaFLOPS	GFLOPS
tera	T	10^{12}	teraFLOPS	TFLOPS
peta	P	10^{15}	petaFLOPS	PFLOPS
exa	E	10^{18}	exaFLOPS	EFLOPS

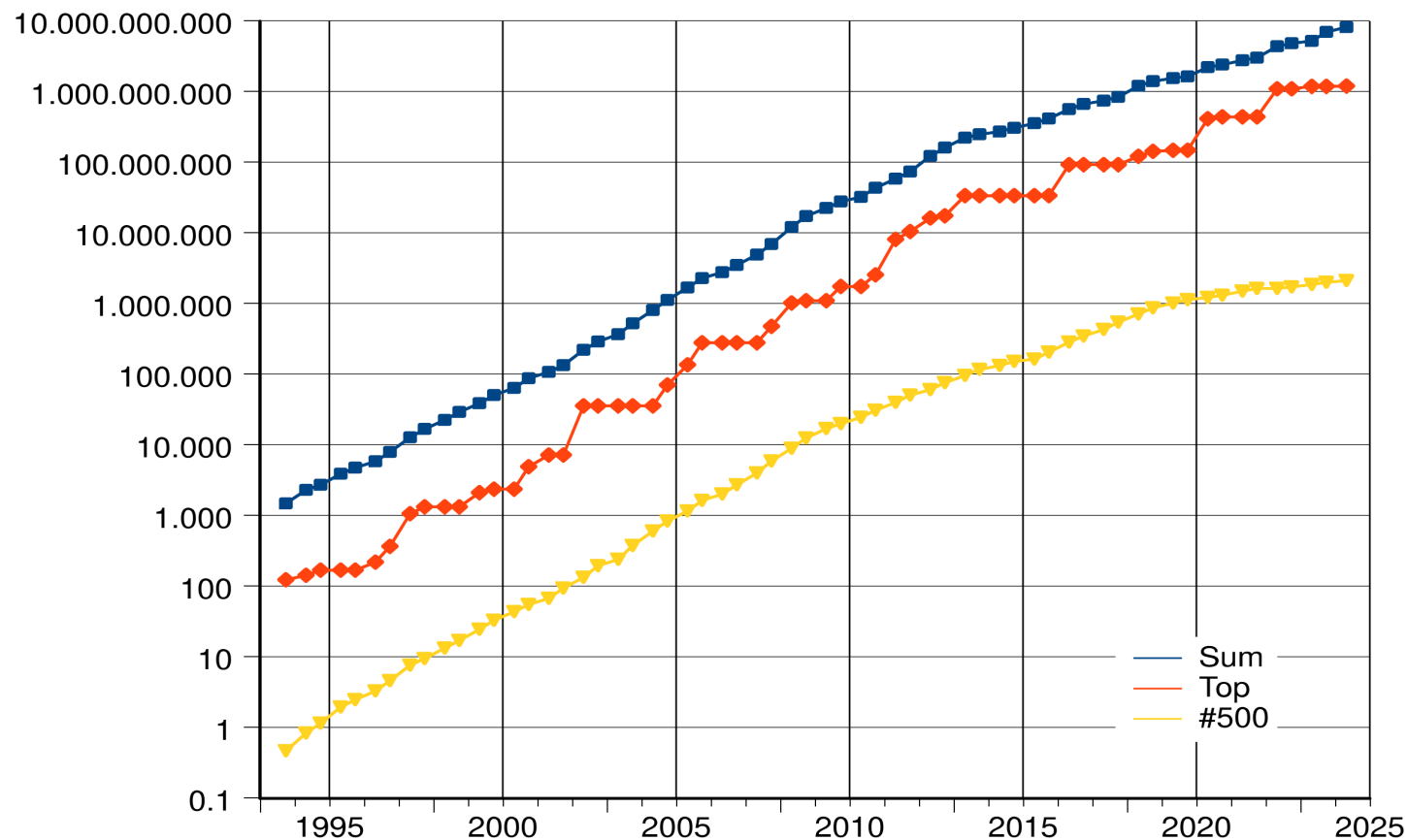
Standard Benchmarks in HPC

- **FLOPS alone insufficient**
 - ignores memory, I/O, algorithmic bottlenecks
- **Community uses benchmark applications simulating realistic workloads**
- **Most important: HPL (Highly Parallel Linpack)**
 - Solves large dense systems of linear equations
 - Basis of **TOP500 list** (updated twice/year)

Standard Benchmarks in HPC

- **Top500 list, adapted from Wikipedia (June 2025)**

The logarithmic y-axis shows performance in FLOPS.



Standard Benchmarks in HPC

■ Top500 List

- Shows exponential growth in performance since 1993
- Tracks: sum of top 500, #1 system, #500 system
- Beyond performance
 - architecture trends, GPU use, software stack, geography, ...
- June 2025 results:
 - #1 El Capitan (LLNL, USA) → 1.742 EFLOPS
 - #2 Frontier (1.353 EFLOPS)
 - #3 Aurora (1.012 EFLOPS)
 - Europe: JUPITER Booster (793.4 PFLOPS, preliminary)
 - Cloud HPC: Microsoft Azure Eagle (561 PFLOPS, rank 5)
 - **MareNostrum 5:**
 - ACC partition (GPUs) → rank 14, 175.3 PFLOPS
 - GPP partition (CPUs) → rank 45, 40.1 PFLOPS
- Trend: dual CPU-only + GPU-accelerated partitions

Standard Benchmarks in HPC

- Next Top500 List:



(EU update)

Grand ceremony marks the launch of the Exascale Supercomputer JUPITER

5 September 2025

JUPITER was unveiled to the public today at Forschungszentrum Jülich in front of high-ranking guests from politics, science, and industry. The German Chancellor Friedrich Merz and the Minister-President of North Rhine-Westphalia Hendrik Wüst attended the inauguration ceremony of Europe's fastest and most energy-efficient supercomputer.



(EU update)

JÜLICH
Forschungszentrum

Institute for Advanced Simulation (IAS)

Jülich Supercomputing Centre (JSC)

News Research Systems Services Education About us

🏠 / JUPITER - Exascale for Europe

JUPITER | The Arrival of Exascale in Europe

Forschungszentrum Jülich will be home to Europe's first exascale computer - called JUPITER.

The supercomputer is set to be the first in Europe to surpass the threshold of one quintillion ("1" followed by 18 zeros) calculations per second.

Peak vs. Sustained Performance

- **Rpeak**

- theoretical max performance (hardware potential)

- **Rmax**

- measured by HPL (real execution, incl. latencies & overheads)

- **TOP500 ranks by Rmax**

- Real applications may perform worse than HPL results
- Important caveat for production environments (application performance often varies significantly depending on the specific workload characteristics)

Energy Efficiency: Green500 List

- **Performance alone insufficient**
→ **energy efficiency crucial**
- **Green500 = FLOPS per watt ($R_{max} \div \text{power}$)**
 - Derived from TOP500 data (no new benchmark)
- **Power budgets = major challenge for exascale**
- **High performance $\neq$ low efficiency → many top systems excel in both**
- **Innovations: heterogeneous CPU–GPU designs, power management, cooling**

Complementary Benchmarks

- **Capture aspects not measured by HPL:**
 - **HPCG**: memory access & communication bottlenecks
 - **STREAM**: sustainable memory bandwidth
 - **Graph500**: irregular data structures, graph analytics, ...
 - ...



Measuring and Understanding Parallel Performance

Parallel Performance Metrics

- **Basic measure: Execution time (wall clock time):**
Elapsed time from first processing unit start to last finish.
- **Metrics to compare parallelization strategies:**
 - Speedup
 - Efficiency
 - Scalability (Strong vs Weak)

Speedup

- **Formula:**

$$Speedup(P) = \frac{T_{serial}}{T_P}$$

- **Goal: Divide work equally, no extra work → ideally**

$$Speedup(P) = P$$

- **Linear speedup: one-to-one performance gain.**

Speedup

■ Example:

- $T_{serial} = 20s$, 90% parallelizable
- Parallel runtime: $T_p = \frac{18}{p} + 2$
- Speedup:

$$Speedup = \frac{20}{18/p + 2}$$

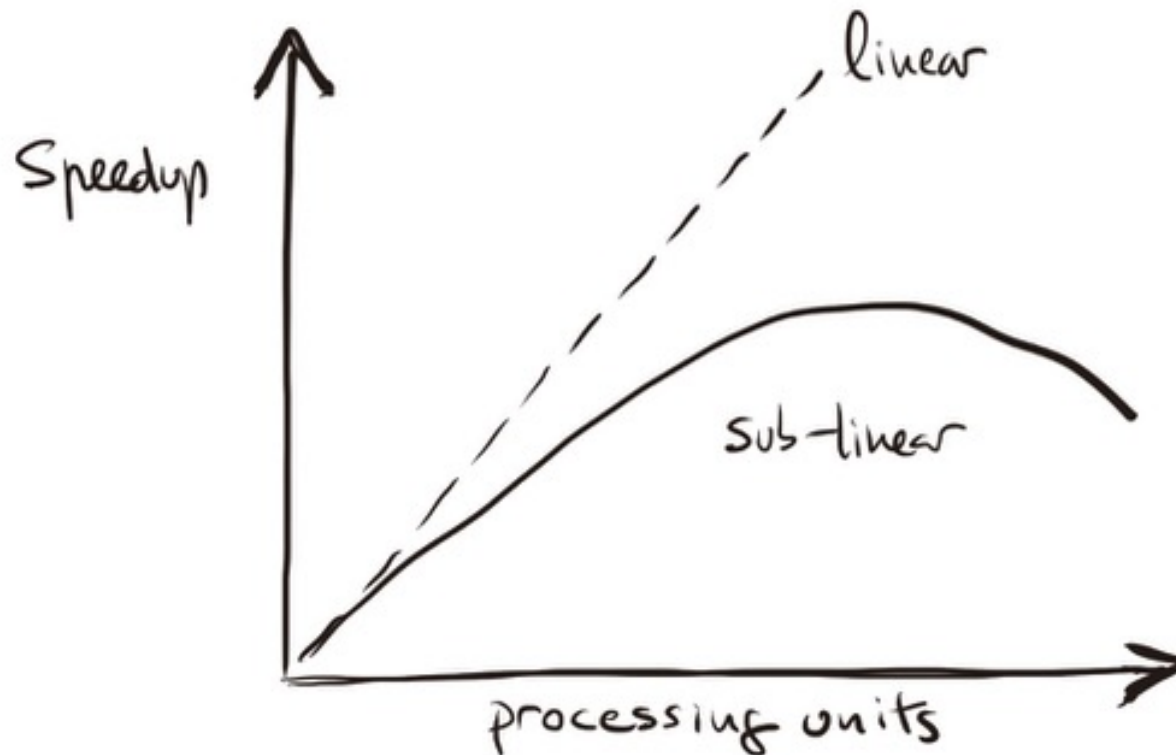
■ As $p \rightarrow \infty$, max speedup = 10 (not 20)

This illustrates that even if we have perfectly parallelized 90% of the program, and even if we have the whole Marenstrum 5 supercomputer to run this code, we will never get a speedup better than 10.

Speedup

■ Sub-linear speedup

Generally, as p increases, we expect that the speedup becomes a smaller and smaller fraction of the ideal linear speedup p . We refer to this case as sub-linear speedup, normally due to different **performance degradation factors**.



Performance Degradation Factors

- **Starvation:** Not enough parallel work → idle processors.
- **Latency:** Waiting for remote data → stalls.
- **Overhead:** Extra work (sync, communication).
- **Contention:** Multiple processes competing for same resource.

→ All reduce performance below ideal linear speedup.

Efficiency

- Efficiency or Parallel Efficiency
- Formula:

$$Efficiency(P) = \frac{Speedup(P)}{P}, \quad 0 \leq Efficiency \leq 1$$

- Measures quality of parallelization.

Efficiency

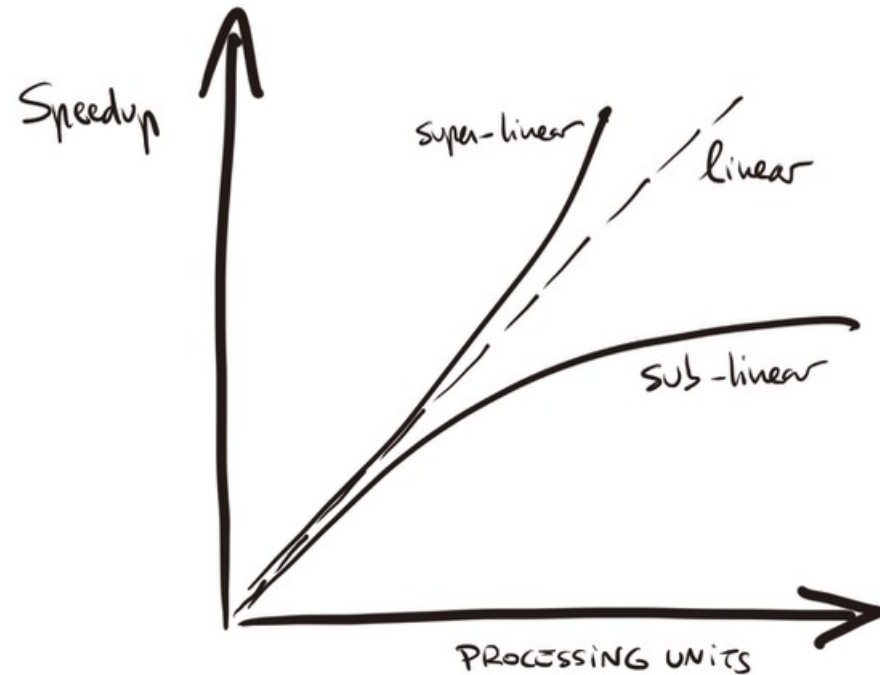
- **Example:**

- $Speedup(32) = 10, P = 32$

- $\frac{10}{32} = 0.3125 \rightarrow$ only 31.25% efficiency.

super-linear speedup?

- **Super-linear speedup:**



- **Super-linear speedup possible (efficiency > 1) due to:**

- Better cache use
- Different algorithms
- Compilation optimizations
- ...

Scalability

- **Two main types**
 - Strong scalability
 - Weak scalability
- **Strong scalability:** Fixed problem size, more processors → shorter time.
 - Example: 120s → 18s with 8 processors
 - Speedup = 6.7 (ideal = 8), Efficiency $\approx 84\%$
- **Weak scalability:** Problem size grows with processors → constant time.
 - Example: 1M cells (120s) vs 8M cells on 8 processors (135s)
 - Efficiency = $120/135 \approx 89\%$

■

Task 1.2 – Understanding Strong and Weak Scaling

■ Case 1: Strong Scaling (fixed-size problem)

- Execution times (sec): 120, 70, 40, 28, 25
- $\text{Speedup}(P) = T_1 / T_p$
 - $P=1 \rightarrow 120/120 = 1$
 - $P=2 \rightarrow 120/70 \approx 1.71$
 - $P=4 \rightarrow 120/40 = 3.0$
 - $P=8 \rightarrow 120/28 \approx 4.29$
 - $P=16 \rightarrow 120/25 = 4.8$
- $\text{Efficiency}(P) = \text{Speedup} / P$
 - $P=2 \rightarrow 0.86$
 - $P=4 \rightarrow 0.75$
 - $P=8 \rightarrow 0.54$
 - $P=16 \rightarrow 0.30$
- Observation:
 - Good speedup initially, but diminishing returns beyond $P=8$
 - Likely due to communication, synchronization, or imbalance

Task 1.2 – Understanding Strong and Weak Scaling

■ Case 2: Weak Scaling (problem size $\uparrow$ with P)

- Execution times (sec): 120, 125, 132, 140, 160
- Problem size $\uparrow$ with P
- Efficiency $\approx T_1 / T_p$
 - $P=2 \rightarrow 120/125 = 0.96$
 - $P=4 \rightarrow 120/132 \approx 0.91$
 - $P=8 \rightarrow 120/140 \approx 0.86$
 - $P=16 \rightarrow 120/160 = 0.75$
- Observation:
 - Runtime increases slightly as P grows
 - Efficiency remains relatively high $\rightarrow$ good weak scaling
 - Degradation can be explained by communication overhead, contention, memory latency, etc.

Scalability

■ Some takeaways:

- Strong scaling → faster with fixed problem size.
- Weak scaling → same runtime with proportionally larger problems.
- Good strong $\neq$ good weak (different bottlenecks).

Speedup Bounds and Scalability Models

- **Two classical models for speedup limits in HPC:**
 - **Amdahl's Law** (1967)
 - **Gustafson-Barsis's Law** (1988)
- **Amdahl: fixed-size problem → speedup is bounded.**
- **Gustafson: growing problem size → speedup can scale nearly linearly.**

Amdahl's Law

- Let f = sequential fraction (cannot be parallelized).

- Maximum speedup with p processors:

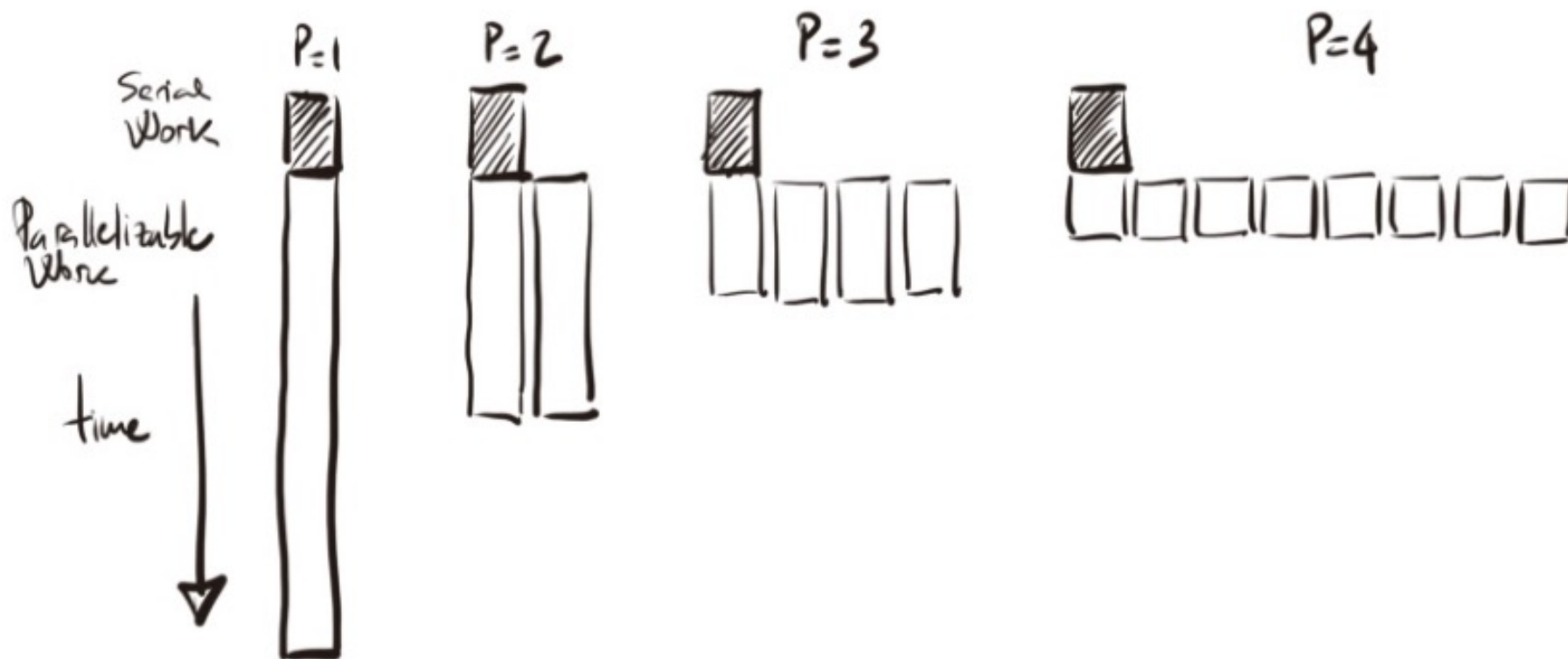
$$Speedup(p) = \frac{1}{f + \frac{(1-f)}{p}}$$

- Even with infinite processors, maximum speedup is:

$$S_{max} = \frac{1}{f}$$

Amdahl's Law

- Intuition visual example:



Amdahl's Law

- **Example**

- 90% parallel, 10% sequential, $p = 8$:

$$\text{Speedup}(8) = \frac{1}{0.1 + 0.9/8} \approx 4.7$$

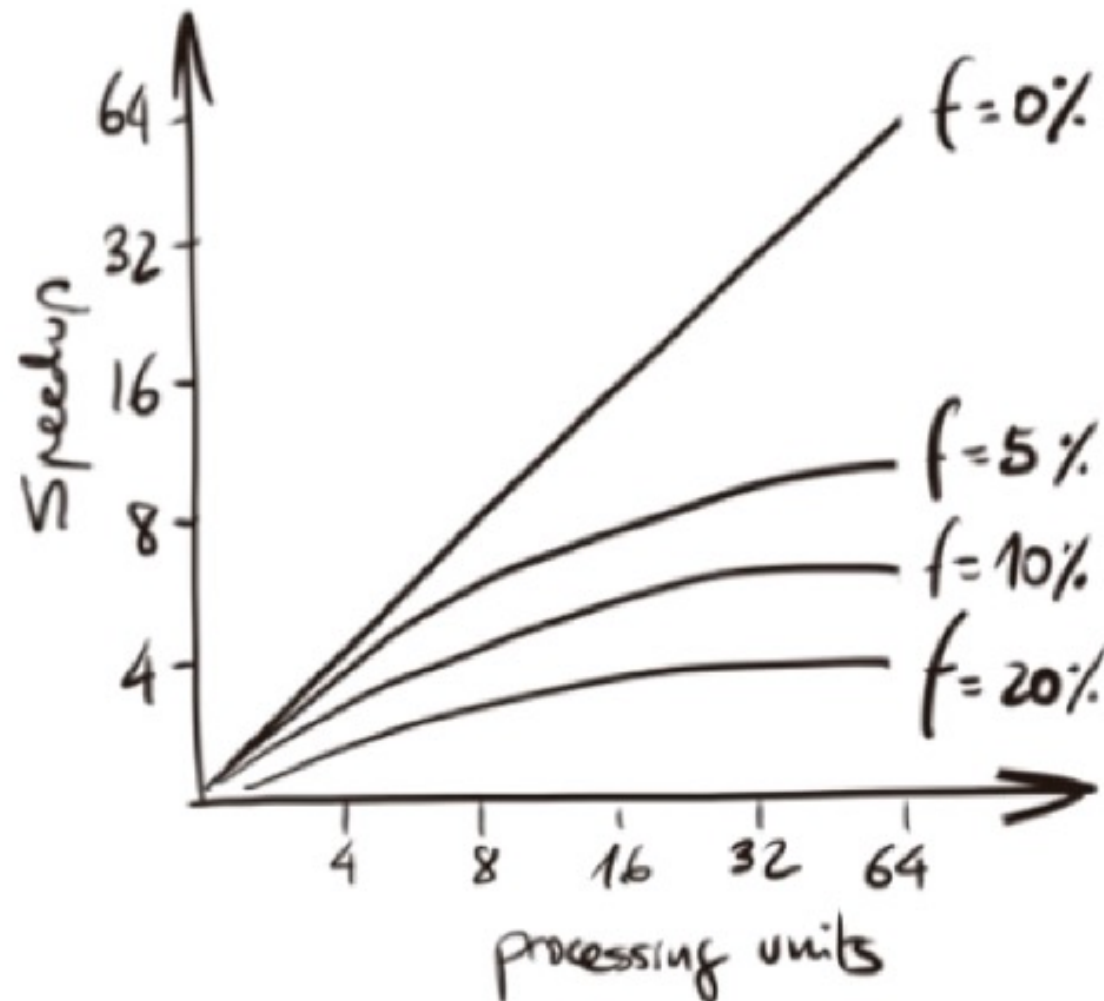
- **Limit with infinite processors:**

$$\lim_{p \rightarrow \infty} \frac{1}{f + \frac{1-f}{p}} = \frac{1}{f}$$

$$\text{Speedup}(\infty) = \frac{1}{0.1} = 10$$

Amdahl's Law

- Takeaway: sequential fraction sets the ceiling.



Amdahl's Law

■ Example 2: Matrix-Vector Multiplication:

- The serial part takes 49.13 microseconds, while the parallel portion takes 2.90 microseconds.

→ Total time: $49.13 + 2.90 = 52.03 \mu s$

- Sequential fraction:

$$f = \frac{49.13}{52.03} \approx 0.944$$

- Maximum speedup with infinite processors:

$$S_{max} = \frac{1}{0.944} \approx 1.059$$

- ## ■ Important takeaway: parallelization has almost no benefit when most of the code is sequential.

Limitations of Amdahl's Law

- **Assumes:**
 - Fixed-size problem, identical processors, linear speedup.
- **Does not capture modern systems:**
 - Heterogeneous architectures (CPU + GPU).
 - Cache/memory effects, compiler optimizations.
 - Different algorithms may achieve better-than-expected results.
- **Still: fundamental model and a good starting point for students.**

Gustafson-Barsis's Law

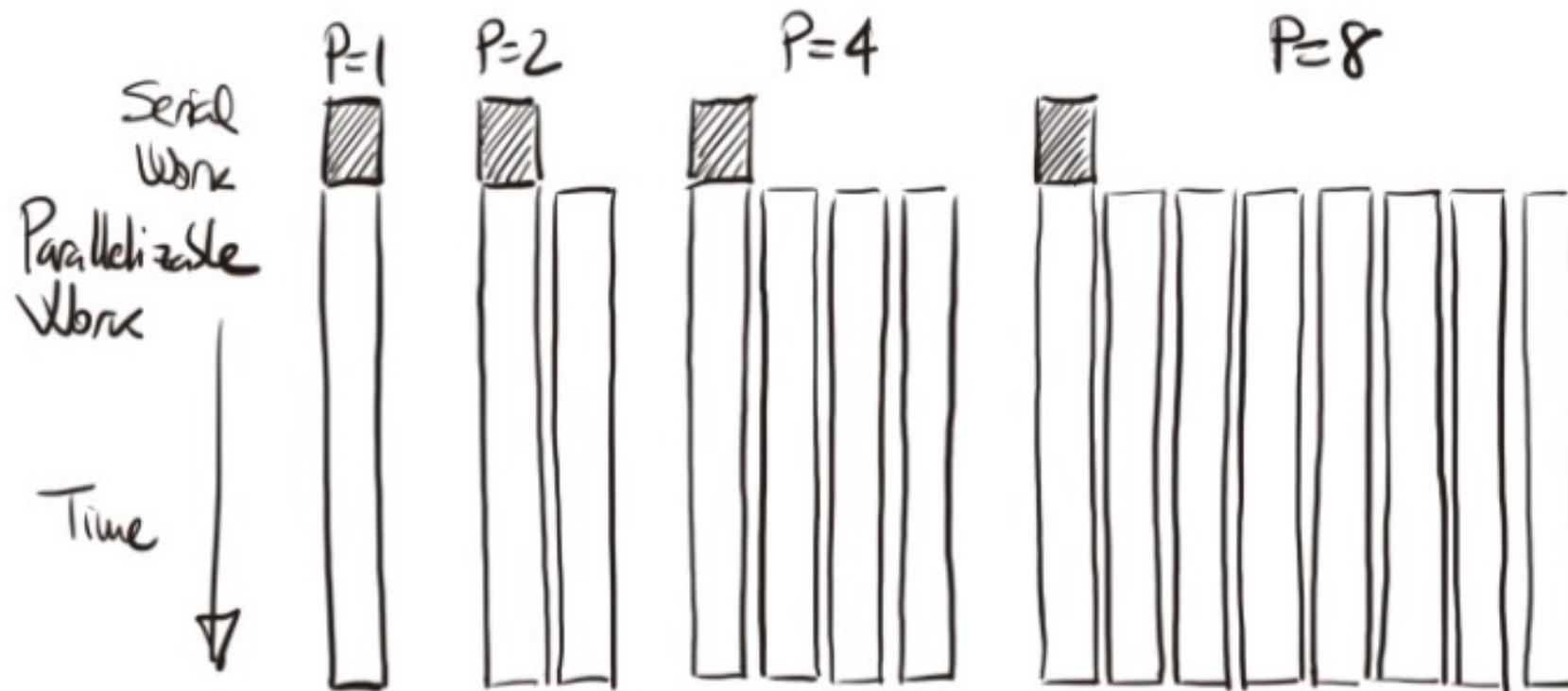
- **More realistic perspective:**
 - As problem size grows, parallel portion dominates.
- **Maximum speedup:**

$$\text{Speedup}(p) = p - s \cdot (p - 1)$$

Where s = serial fraction.

Gustafson-Barsis's Law

- Intuition visual example:



Gustafson-Barsis's Law

■ Example:

- Matrix-vector multiplication on 32 processors.
- The benchmarking indicates that 37% of this time is spent executing the serial portion of the computation on a single processing unit $\rightarrow s = 0.37$

■ Gustafson-Barsis's Law

$$Speedup(32) = 32 - 0.37 \cdot (32 - 1) \approx 20.53$$

■ Much higher than Amdahl's prediction

$$Speedup(32) = \frac{1}{0.37 + \frac{0.63}{32}} \approx 2.57$$

Parallel Overhead

- **Real systems deviate from ideal models due to:**
 - **Communication overhead.**
 - **Synchronization overhead.**
 - **Load imbalance.**
 - **Startup overhead** (thread/process initialization).
 - **Memory & cache effects.**
 - **OS and runtime overheads.**
- **IMPORTANT Conclusion: real speedup is always less than theoretical.**

These slides are based on the book
Supercomputing for Artificial Intelligence (Torres, 2025).
more info: <https://torres.ai/hpc4aibook/>

PDF slides are freely available for students.

Teachers using this book may request the PPTX version for
classroom use.